Lesson 6 - Filter Analysis

Now that we have some familiarity with MATLAB, we can start applying it to actually analyze some filters and signals (as the course name would suggest).

Note: Make sure you have the Signal Processing Toolbox installed if you are using the app version of MATLAB.

Discrete Fourier Transform

Typically, we would like to use the Fourier transform to find the complex exponential "components" of a typical continuous signal. Unfortunately, our computers can't exactly represent infinitely many points to infinite precision. Instead, we will use the Discrete Fourier Transform (or DFT) for discrete-time, discrete-valued systems which is defined for a signal x[n] as

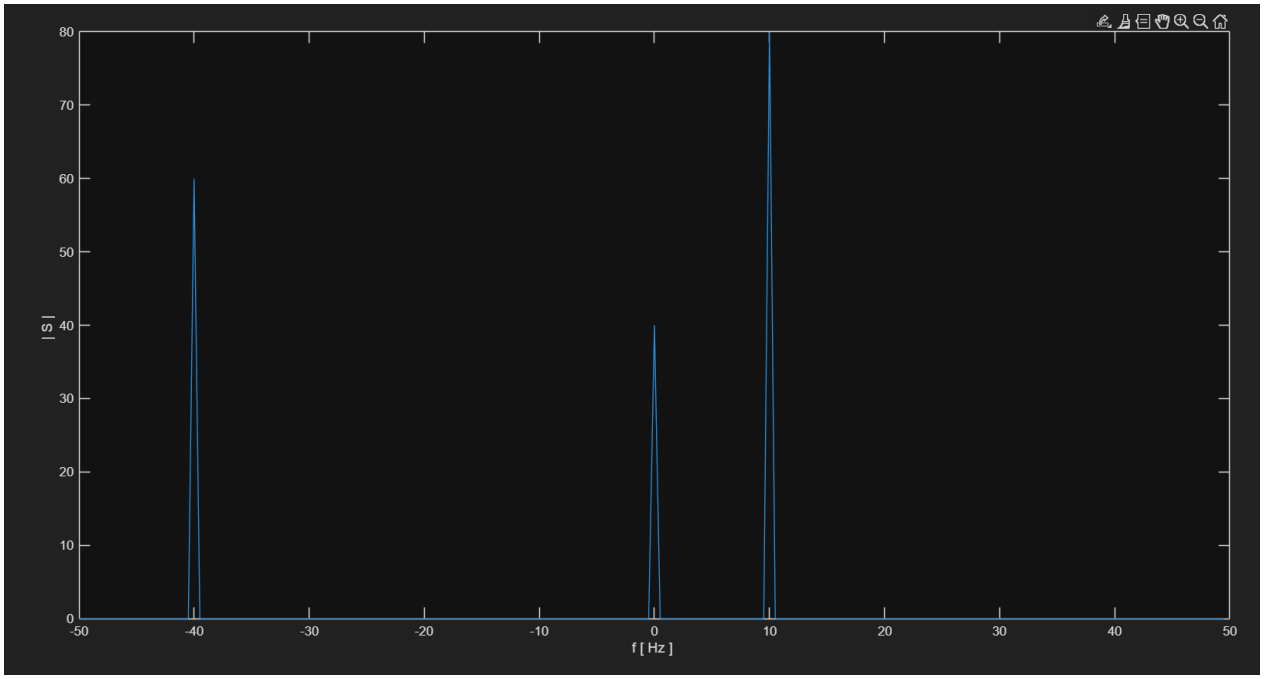
$$X[k] = \sum_{m=0}^{n-1} x[m] e^{\frac{-2\pi i k \cdot m}{n}}.$$

The equation above allows us to compute a n-point DFT of a signal. The value of n allows us to vary our resolution in the frequency domain with so-called bin frequencies with the following bin spacing: $\Delta\omega = \frac{2\pi}{n}$

The Fast Fourier Transform (FFT) is nothing more than an optimization of the DFT by removing redundant computations. In MATLAB, we can compute the DFT with the fft() function:

```
fs = 100;
T = 1 / fs;
l = 200;
t = (0:l- 1) * T;
s = 0.2;
s = s + 0.4 * exp(1j * 2 * pi * 10 * t);
s = s + 0.3 * exp(-1j * 2 * pi * 40 * t);
S = fftshift(fft(s));
figure;
plot(fs / l * (-l / 2:l / 2- 1), abs(S));
ylabel('|S|');
xlabel('f [Hz]');
```

Due to how the DFT is defined, the output of fft() will be DC, followed by positive and negative frequencies. As such, it's typically desirable to center our output around DC with fftshift(). Other things to note are that the output may be complex-valued so we typically plot the magnitude of the FFT and that the frequency range is from $-f_s/2$ to $f_s/2$ where f_x is the sampling rate.



z-Transforms

The z-transform is defined for a discrete-time signal $x[n]$ as

$$X(z) = \sum_{n=-\infty}^{\infty} x[n]z^{-n}$$

.

This allows us to convert any discrete time signal into a complex representation in the z-domain. The z-transform itself has some nice properties such as

- Linearity: $c_1x_1 + c_2x_2 \longleftrightarrow c_1X_1 + c_2X_2$
- Convolution: $h[n] * x[n] \longleftrightarrow H \cdot X$
- Delay: $x[n-d] \longleftrightarrow z^{-d}X(z)$

It also lets us analyze the behavior of transfer functions that are rational functions which are commonly represented as

$$H(z) = \frac{b_0 + b_1z^{-1} + ... + b_mz^{-m}}{a_0 + a_1z^{-1} + ... + a_n^{-n}}$$

or equivalently as

$$H(z) = K\frac{(z-z_1)^{n_1}...(z-z_N)^{n_N}}{(z-p_1)^{m_1}...(z-m_M)^{m_M}}$$

where K is the gain of the system, z_i are the zeros of the system, and p_i are the poles of the system.

The second form is preferred over the raw polynomial coefficients as it is less sensitive to the issue of roundoff error inside the computer.

Converting between Transfer Function Forms

As noted above, there are two main ways to represent a transfer function in the z-domain (the expanded polynomial form and the pole-zero-gain form) with the pole-zero-gain form being used the most. For example, if I had the transfer function

$$H(z) = \frac{(z+3)(z-1)^2}{z(z-3)}$$

which is equivalently

$$\frac{1 + z^{-1} - 5z^{-2} + 3z^{-3}}{1 - 3z^{-1}},$$

We can use the following code snippet to switch back and forth between the two forms.

```
z = [-3; 1; 1];
p = [0; 3];
k = 1;

% Note: zeros, poles are represented as columns
% Polynomial coefficients are row vectors

[b,a] = zp2tf(z, p, k);
[z0, p0, k0] = tf2zpk(b, a);
```

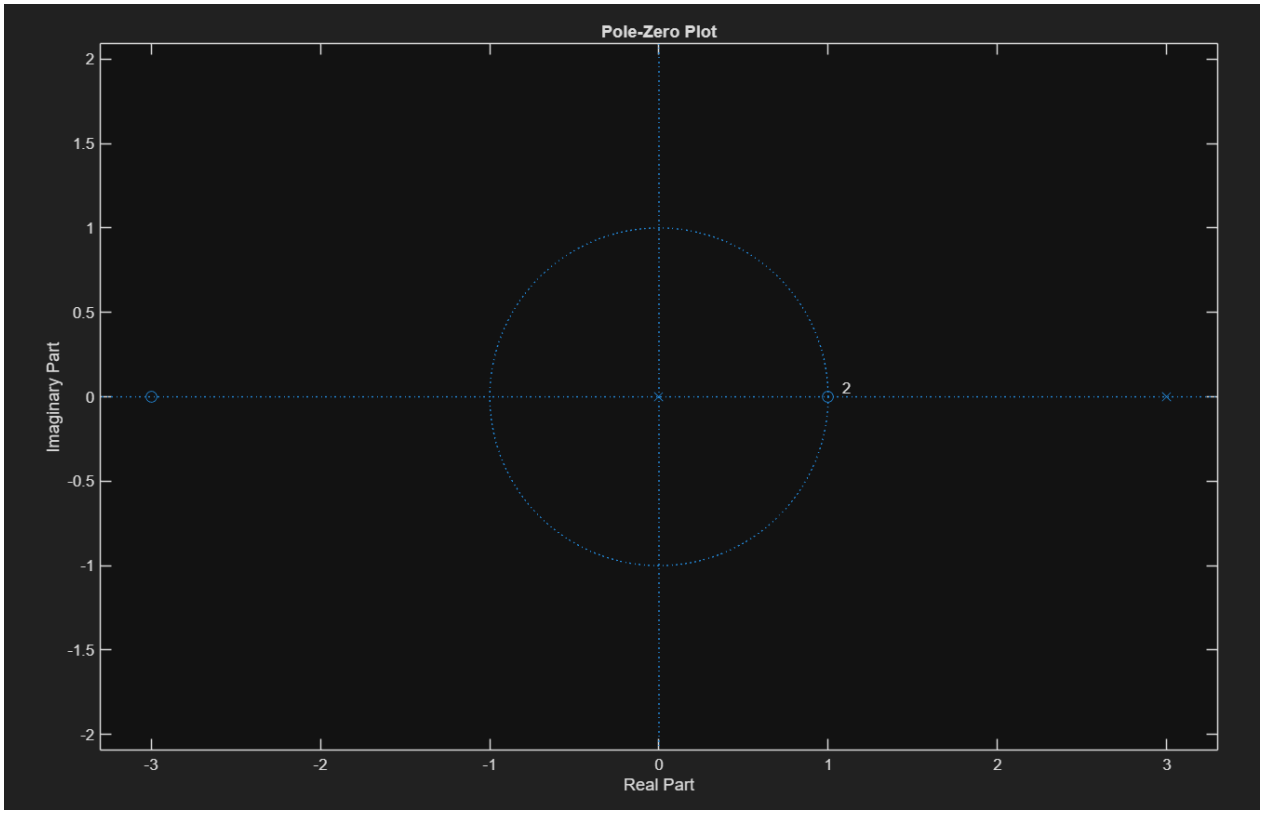
In most applications for signal processing, you would typically switch the raw coefficients for the zero-pole-gain form for most calculations and then convert back later if you need to display the transfer function in polynomial form.

Plotting Zeros and Poles

If we want to plot the zeros and poles of our digital filter, we can use the function zplane() to plot our points. This will typically plot the unit circle along with the poles, zeros, and their multiplicities in the complex plane.

MATLAB does not currently have a function in the Toolbox for the analog version for zplane (for Laplace transforms), but it is certainly possible to implement your on version of the function with what you have learned so far in the plotting lesson.

```
figure;
zplane(z, p);
```



Frequency Response

In order to find out how the frequency of a signal would affect the resulting output magnitude and phase, we can use freqz() to plot the frequency response.

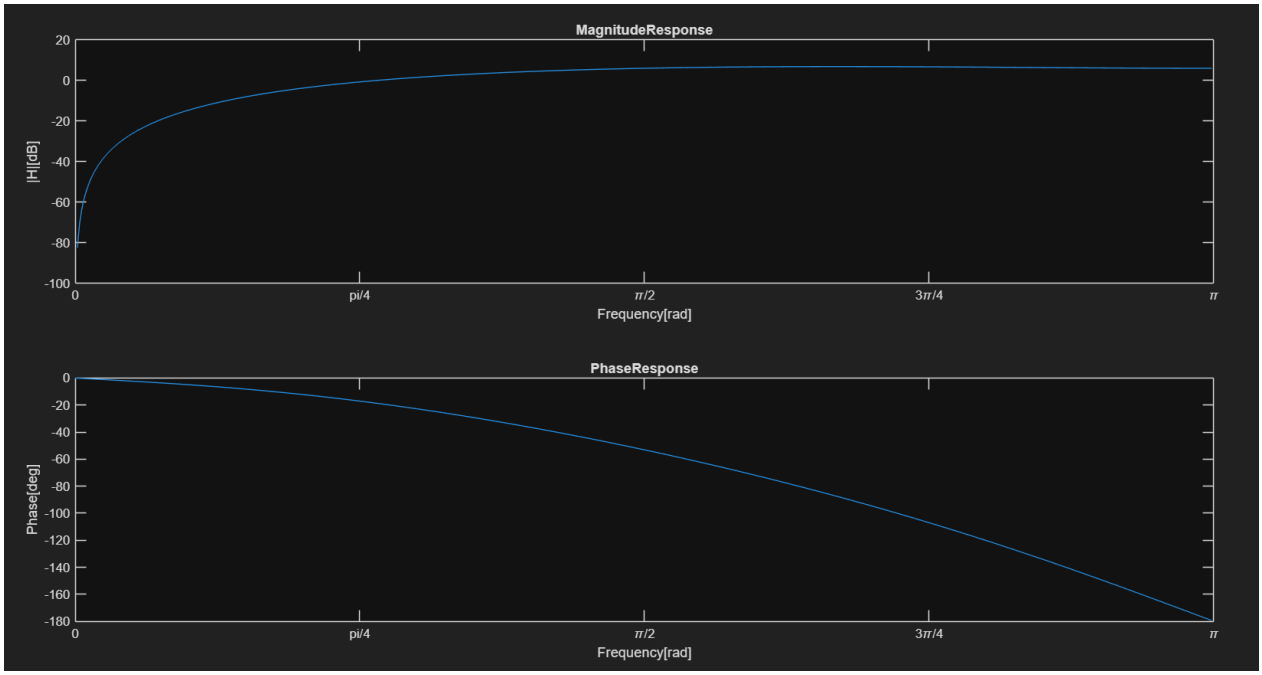
Note: Sometimes, there are jump discontinuities in the phase response due to the magnitude of the output going to 0. So, we use unwrap() to help resolve this and to let the angle go beyond ±180 degrees.

```
[H, w]= freqz(b,a);

H_dB = 20* log10(abs(H));
H_ph = rad2deg(unwrap(angle(H)));

figure;

subplot(2, 1, 1);
plot(w,H_dB);
title('MagnitudeResponse');
xlim([0, pi]);
xticks([0, pi/ 4, pi/ 2, 3 *pi /4,pi]);
xticklabels({'0','pi/4','\pi/2', '3\pi/4', '\pi'});
xlabel('Frequency[rad]');
ylabel('|H|[dB]');

subplot(2, 1, 2);
```

```
plot(w,H_ph);
title('PhaseResponse');
xlim([0, pi]);
xticks([0, pi/ 4, pi/ 2, 3 *pi /4,pi]);
xticklabels({'0','pi/4','\pi/2', '3\pi/4', '\pi'});
xlabel('Frequency[rad]');
ylabel('Phase[deg]');
```